Gemini
MIPS Archit...
MIPS Architecture
PPTX
Show thinking
Export to Sheets
Code snippet
Sources
Program Co...ubroutines
PPTX
Show thinking
Export to Sheets
Real Time ...nterfacing
PPTX
Show thinking
Export to Sheets
Sources
Software Interrupts
PPTX
Show thinking

make easiest notes of this slide

Based on the presentation slides, here are the easiest, most organized notes for MIPS Architecture.

1. What is MIPS?

MIPS (Microprocessor without Interlocked Pipeline Stages) is a processor design that follows the **RISC** (Reduced Instruction Set Computer) philosophy.

- **RISC Philosophy:** Keep it simple. It uses a small set of uniform instructions to run faster.
- **Key Design Rules:**
  - **Fixed Size:** All instructions are exactly 32 bits long.
  - **Load/Store Architecture:** You cannot do math directly on memory. You must **Load** data into a register, do the math, and then **Store** it back.
  - **Pipelining:** It executes instructions in assembly-line stages (Fetch, Decode, Execute, Memory, Write Back) to increase speed.

---

2. The Registers (The CPU's "Hands")

MIPS has **32 general-purpose registers**, each 32 bits wide. They are the fastest storage in the computer.

Common Registers & Their Jobs:

Register Name	Number	Usage	Convention
$zero	$0	Always equals 0. Cannot be changed.	-
$v0 - $v1	$2-3	Used to return values from a function.	Caller Saves
$a0 - $a3	$4-7	Used to pass arguments (inputs) to a function.	Caller Saves
$t0 - $t9	$8-15	**Temporary** variables. Can be overwritten easily.	Caller Saves
$s0 - $s7	$16-23	**Saved** variables. Must be preserved.	Callee Saves
$sp	$29	**Stack Pointer**. Points to the top of the stack.	-
$ra	$31	**Return Address**. Remembers where to go back after a function finishes.	-

---

3. Instruction Formats (The Language)

MIPS instructions are divided into three specific formats based on what they do.

- **R-Type (Register Type):** Used for math and logic between registers.
  - *Format:* `Opcode | rs | rt | rd | shamt | funct`
  - *Example:* `ADD $t0, $t1, $t2` (Adds $t1 and $t2, stores result in $t0).
- **I-Type (Immediate Type):** Used for constants (immediate numbers), memory access, and branches.
  - *Format:* `Opcode | rs | rt | Immediate Value`
  - *Example:* `ADDI $t0, $t1, 10` (Adds 10 to $t1).
  - *Example:* `LW $t0, 8($s0)` (Loads word from memory).
- **J-Type (Jump Type):** Used to jump to a different part of the code (like a loop or function).
  - *Format:* `Opcode | Target Address`
  - *Example:* `J 10000` (Jump to address 10000).

---

4. Memory Model

- **Byte-Addressable:** Each address points to 1 byte (8 bits).
- **Word Alignment:** MIPS usually processes **Words** (32 bits / 4 bytes). Therefore, memory addresses for words must be multiples of **4** (e.g., 0, 4, 8, 12) .
- **Memory Layout:**
  - **User Space:** Where your programs run (lower addresses).
  - **Kernel Space:** Where the Operating System runs (higher addresses).
- **MMU (Memory Management Unit):** Uses a **TLB** (Translation Lookaside Buffer) to translate "Virtual Addresses" (what the program sees) into "Physical Addresses" (where data actually is in RAM) .

---

5. Simple MIPS Assembly Example

Here is a program from the slides that adds two numbers found in memory .

```
.data
A: .word 5          # Define variable A = 5
B: .word 7          # Define variable B = 7
C: .space 4         # Reserve space for result C

.text
main:
  lw $t0, A         # Load A from memory into register $t0
  lw $t1, B         # Load B from memory into register $t1
  add $t2, $t0, $t1 # Add $t0 and $t1, store result in $t2
  sw $t2, C         # Store the result ($t2) back into memory at C
  li $v0, 10        # Load code 10 (exit) into $v0
  syscall           # Tell the system to execute the exit
```

Would you like me to create a quick quiz based on these notes to help you prepare for an exam?

help me prepare this slide for exam

Based on standard MIPS architecture topics, here are the **easiest notes** for **Program Control and Subroutines**. These concepts are crucial for understanding how code executes decisions, loops, and functions.

1. Control Flow (Making Decisions)

Just like high-level languages have `if` statements and loops, MIPS uses **branches** and **jumps** to control which instruction runs next.

- **Conditional Branches (The "If"):** These instructions check a condition. If it's true, the computer "branches" (jumps) to a label.
  - `beq $t0, $t1, Label` **(Branch if Equal):** If `$t0 == $t1`, go to `Label`.
  - `bne $t0, $t1, Label` **(Branch if Not Equal):** If `$t0 != $t1`, go to `Label`.
- **Set Less Than (The Comparison):** MIPS doesn't have `<` or `>` symbols in branches. It uses `slt` to compare.
  - `slt $t0, $s1, $s2` **(Set Less Than):**
    - If `$s1 < $s2`, then `$t0 = 1`.
    - Otherwise, `$t0 = 0`.
  - *Usage:* You usually follow `slt` with `bne` or `beq` to make a decision based on the comparison.

---

2. Loops (Repeating Code)

Loops (like `while` or `for`) are built using a combination of **Conditional Branches** (to check if the loop is done) and **Unconditional Jumps** (to go back to the start).

- `j Label` **(Unconditional Jump):** Always jump to `Label`. Used to restart a loop.

Logic Flow:

1. Check condition (using `bne` / `beq`). If false, jump to "Exit".
2. Run loop body.
3. `j Loop_Start` (Jump back to the top).

---

3. Subroutines (Functions/Methods)

Subroutines are blocks of code you can call from anywhere. MIPS uses specific instructions to handle moving between the "Caller" (main program) and the "Callee" (the function).

- `jal Label` **(Jump and Link):**
  - **Jump:** Go to the function at `Label`.
  - **Link:** Save the address of the *next* instruction (the return address) into register `$ra` (Return Address). This remembers where to come back to.
- `jr $ra` **(Jump Register):**
  - Look at the address stored in `$ra` and jump back there. This acts as the **return** statement.

---

4. The Stack (Memory Storage)

Registers are fast but limited. If a function needs more space (e.g., to save `$s` registers or call *another* function), it uses the **Stack**.

- `$sp` **(Stack Pointer):** A register that keeps track of the "top" of the stack in memory.
- **Pushing (Saving):** To save data, you **subtract** from `$sp` (grow down) and `sw` (store word).

```
addi $sp, $sp, -4  # Make space for 1 word
sw   $s0, 0($sp)   # Save $s0 to the stack
```

- **Popping (Restoring):** To restore data, you `lw` (load word) and **add** to `$sp` (shrink up).

```
lw   $s0, 0($sp)   # Restore $s0
addi $sp, $sp, 4   # Clear the space
```

- **Nested Calls:** If Function A calls Function B, Function A **must** save its `$ra` on the stack before calling B, or else the return address will be overwritten!

---

5. Key Registers for Subroutines (Recap)

Register	Name	Role
$a0 - $a3	Arguments	Inputs passed *into* the function.
$v0 - $v1	Values	Outputs returned *from* the function.
$ra	Return Address	The "bookmark" to get back to the caller.
$sp	Stack Pointer	Location of the current stack top.

Would you like a practice quiz on Subroutines and Branching to test your knowledge?

assist me prepare this slide for exam

Based on the "Real Time Interrupts and Hardware Interfacing" presentation, here are the **easiest notes** to help you prepare for your exam.

1. What is an Interrupt?

An interrupt is a signal sent to the processor that tells it to **stop what it is doing** and handle a high-priority event immediately.

- **Analogy:**
  - **Polling:** Checking the oven every 30 seconds to see if your pizza is done. (Waste of time/energy).
  - **Interrupt:** Waiting for the timer to ring. (Efficient, you can do other things while waiting) .

---

2. Interrupts vs. Polling

The CPU needs to communicate with external hardware (like a keyboard or sensor). There are two ways to do this:

Feature	Polling	Interrupts
How it works	CPU constantly checks the device status in a loop.	Device sends a signal to CPU only when ready.
CPU Usage	High (wastes cycles checking).	Low (CPU sleeps or does other work).
Efficiency	Inefficient for rare events.	Very efficient.
Response Time	Depends on when the loop checks next.	Immediate (Real-Time).

---

3. The Interrupt Life Cycle

When an interrupt occurs, the CPU follows a strict sequence:

1. **Suspend:** Pause the current program and save its location (PC) and status .
2. **Jump:** Go to a specific address called the **Interrupt Vector** .
3. **Execute:** Run the **ISR** (Interrupt Service Routine) – the code that handles the specific event.
4. **Restore:** Retrieve the saved status and location.
5. **Return:** Go back to the exact spot where the original program left off (`reti` or return from interrupt instruction) .

---

4. Key Terminology

- **ISR (Interrupt Service Routine):** The function or small program that runs when the interrupt happens (e.g., the code that reads a key when you press a keyboard button) .
- **IVT (Interrupt Vector Table):** A table in memory that holds the addresses (locations) of all the ISRs. It tells the CPU *where* to go for each type of interrupt .
- **Maskable Interrupt:** An interrupt you can **ignore** (disable) if you are doing something critical (e.g., "Do Not Disturb" mode) .
- **Non-Maskable Interrupt (NMI):** An interrupt that **cannot** be ignored (e.g., Low Battery/Power Failure).

---

5. Real-Time Considerations

In Real-Time Systems (like braking systems in cars), timing is critical.

- **Latency:** The time gap between the interrupt signal arriving and the ISR actually starting. This must be kept very short.
- **Determinism:** The system must guarantee that the interrupt is handled within a known, fixed time .

Would you like me to generate a practice quiz on these concepts (Interrupts vs Polling, ISRs) to test your readiness?

assist me prepare this slide or exam

Based on the "Software Interrupts" presentation, here are the **easiest notes** to help you study.

1. What is a Software Interrupt?

Unlike hardware interrupts (which come from external devices like a keyboard), a **Software Interrupt** is generated by the **program itself** running inside the processor.

- It acts like a special function call (extended far call) to get the processor to do a specific task.
- **Key Instruction:** `INT n` (where `n` is a number from 0–255).

---

2. How It Works (The Mechanism)

When the code executes `INT n`, the processor follows these steps automatically:

1. **Save State:** Pushes the Flags, Code Segment (CS), and Instruction Pointer (IP) onto the stack to remember where it was.
2. **Find Handler:** Looks up the address of the **ISR** (Interrupt Service Routine) in the **Interrupt Vector Table (IVT)**.
   - *Note:* The IVT is located at memory address `0` and is 1 KB in size.
3. **Execute:** Runs the ISR code.
4. **Return:** When finished, the ISR uses the `IRET` instruction to pop the saved values from the stack and return to your program .

---

3. Important Reserved Interrupts (Memorize These)

The first few interrupts (0–4) are reserved for specific errors or debugging tools .

Interrupt	Name	Cause
INT 0	Divide by Zero	Triggered if you try to divide a number by 0.
INT 1	Single Step (Trap)	Used by debuggers to run code one line at a time.
INT 2	NMI (Non-Maskable)	Critical hardware failures (cannot be ignored).
INT 3	Breakpoint	Used by debuggers to pause the program.
INT 4	Overflow	Triggered by the `INTO` instruction if a math result is too big.

Export to Sheets

---

4. BIOS vs. DOS Interrupts

These are pre-written "service packs" built into the computer so you don't have to write code to control hardware directly .

- **BIOS Interrupts (Low-Level Hardware):**
  - **INT 10h:** Video services (e.g., clear screen, print text).
  - **INT 16h:** Keyboard input.
- **DOS Interrupts (OS Services):**
  - **INT 21h:** The "Master" switch for DOS. You put a **Service Number** in register `AH` to tell it what to do (e.g., Read file, Print string, Exit program) .

---

5. Example Code: Printing "Hello World"

This code snippet from your slides uses **BIOS (INT 10h)** to print text.

Code snippet

```
mov ah, 0x13     ; Service 13h: Print String
mov al, 1        ; Sub-service: Update cursor after printing
mov bl, 7        ; Color attribute (Standard white on black)
mov cx, 11       ; Length of string ("Hello World" is 11 chars)
mov bp, message  ; Where the string is located in memory
int 0x10         ; Call BIOS to do the work!
```

Would you like me to create a "Match the Interrupt" quiz (e.g., matching INT 21 to DOS) to check your memory?

Sources

Bit Manipulation
P PPTX

now make step by step notes of this slide

Show thinking

Based on the "Bit Manipulations" slides, here are the **step-by-step notes** organized to make these concepts easy to understand.

1. Why Bit Manipulation?

Processors are digital devices that work with bits (0s and 1s). Bit manipulation allows you to:

- Perform fast math (multiplication/division) without using slow math instructions.
- Control specific hardware signals (like setting a specific pin on a chip).
- Compress data.

---

2. Shifting Instructions (Moving Bits)

Think of a register as a pipe holding balls (bits). Shifting moves them left or right.

- **SHR (Shift Logical Right):**
  - **Action:** Moves bits right. A `0` enters from the left. The bit that falls off the right goes into the **Carry Flag (CF)**.
  - **Use:** Efficient **division by 2** for unsigned numbers.
- **SHL / SAL (Shift Logical/Arithmetic Left):**
  - **Action:** Moves bits left. A `0` enters from the right. The bit falling off the left goes to **CF**.
  - **Use:** Efficient **multiplication by 2** .
- **SAR (Shift Arithmetic Right) - *Important for Signed Numbers*:**
  - **Action:** Moves bits right, but **keeps the old Sign Bit** (MSB) at the left.
  - **Why?** If you have a negative number (starts with 1), regular shifting (SHR) would insert a 0 and turn it positive. SAR preserves the negative sign while dividing by 2 .

---

3. Rotation Instructions (Wrap-Around)

Unlike shifting, where bits "fall off" and are lost (except for the one in Carry), rotation wraps them around.

- **ROR (Rotate Right):** The bit falling off the right acts as the new bit entering the left.
- **ROL (Rotate Left):** The bit falling off the left acts as the new bit entering the right.
- **RCR / RCL (Rotate through Carry):**
  - This includes the **Carry Flag** in the circle.
  - *Analogy:* It treats the register + Carry Flag as a 9-bit (or 17-bit) loop. The bit falling out goes into Carry, and the old Carry value enters the other side .

---

4. Binary Multiplication Algorithm

How does a processor multiply `13 x 5`? It uses the **Shift and Add** method.

The Algorithm:

1. **Check:** Look at the rightmost bit (LSB) of the Multiplier.
2. **Add:** If it is `1`, add the **Multiplicand** to the Result. If `0`, do nothing.
3. **Shift:** Always **Shift the Multiplicand Left** (multiply by 2) and **Shift the Multiplier Right** (to process the next bit).
4. **Repeat:** Do this for every bit in the multiplier .

---

5. Extended Arithmetic (Big Numbers)

What if you need to multiply or add 32-bit numbers on a 16-bit processor? You must break them into chunks.

- **Extended Shifting:**
  - To shift a 32-bit number, you shift the lower 16 bits first (using `SHL`). The bit that "falls off" goes into the Carry Flag.
  - Then, use `RCL` **(Rotate through Carry Left)** on the upper 16 bits to pull that Carry bit in. This stitches the two halves together.
- **ADC (Add with Carry):**
  - Regular `ADD` is for the first chunk.
  - `ADC` adds the two operands **plus** the Carry Flag from the previous addition. This handles the "carry over" from the lower half .
- **SBB (Subtract with Borrow):**
  - Similar to ADC, but subtracts the Carry Flag (borrow) from the result.

---

6. Bitwise Logic & Masking (The "Digital Tape")

These operations allow you to modify specific bits without touching others.

Operation	Logic	Purpose (Masking)	Example
AND	1 only if both are 1	**Clear** bits (force to 0).	`AND AL, 0Fh` (Keeps lower 4 bits, clears upper 4) .
OR	1 if either is 1	**Set** bits (force to 1).	`OR AL, 80h` (Forces the first bit to 1, keeps others) .
XOR	1 if bits are different	**Invert** bits (flip 0 to 1).	`XOR AL, FFh` (Flips every bit in AL).
TEST	Same as AND	**Check** bits.	Updates flags (Zero Flag) without changing the data. Non-destructive .

Export to Sheets

Would you like me to write the "Extended Shifting" code snippet from the slides to help you practice?

Sources

Ask Gemini 3

+ Pro

Gemini can make mistakes, so double-check it